LuminaGarden - Assignment 3 progress report

Kartik Patel | Safa Akbar | October 12, 2026

Plan vs. reality

The Assignment 2 PDF defines the vertical slice as one procedural plant type, click-to-plant, randomized height/branching/color, staged growth from sprout to leaves, emissive materials, one colored point light per plant, and basic bloom. The presentation describes a larger semester project; those additional features were not treated as midterm requirements.

All core slice features are implemented in a Three.js/TypeScript browser application. A ground raycast places seeds. A seeded random generator constrains height to 1.85-2.90 scene units, branch count to 5-8, and leaf count to 3-5 per branch. Branches follow a golden-angle distribution with bounded angle variation. The same seed produces the same geometry. Growth lasts nine seconds: stem segments extend from their bases, then branches, leaves, and petals reveal on separate schedules. This avoids scaling a completed plant as one object.

Leaves use a custom GLSL emission gradient along their vein and toward their tip. Each plant has a colored point light; UnrealBloomPass supplies the halo and OutputPass performs output conversion. Stem, leaf, petal, and surface-stone geometry use instancing. Mature plants stop uploading growth transforms. Bloom and point-light toggles make the rendering contributions visible.

The user requested five plants or fewer, so the application enforces a hard limit of five and starts with three mature examples. The original 15-20-plant target is therefore not validated. Wind, movable lights, day/night, multiple species, and shadows remain deferred as explicitly cut in the MVP. Clear/reset was added as a repeatable testing control. A small seeded PRNG, HTML controls, and a shared growth clock replace the proposed simplex-noise, lil-gui, and GSAP dependencies.

Verification and performance

The production build and four generator tests pass. Three Chrome browser tests pass, covering ground clicks, staged growth, seed replay, the five-plant cap, rendering controls, drag-versus-click, cancellation, and a useful WebGL2 failure. The generator tests examine finite and bounded geometry over 300 seeds. No browser or shader errors were captured in the demo.

Recorded stress test: 10 seconds growth/warm-up and 8 seconds measured at each level, bloom 0.65, one point light per plant.

1 plant(s): 60.0 average FPS; 16.7 ms p95; 23 draw calls; slowest frame 16.8 ms.

3 plant(s): 60.0 average FPS; 16.8 ms p95; 33 draw calls; slowest frame 16.8 ms.

5 plant(s): 60.0 average FPS; 16.7 ms p95; 43 draw calls; slowest frame 16.8 ms.

Environment: ANGLE (Apple, ANGLE Metal Renderer: Apple M2, Unspecified Version); Chrome 155.0.0.0; 1086 x 430 canvas; pixel ratio 1. Recording was active during measurement. Average FPS uses animation-frame intervals, not GPU timer queries. This capture stayed near 60 FPS at the three measured workloads; it does not guarantee that result at other resolutions or on other GPUs. Five plants do not establish the original 20-plant goal. Raw data: docs/benchmark-results.json.

AI utility

AI assistance was used to structure the procedural rules, author the leaf emission GLSL, implement controls and profiling, and create verification scripts. It helped produce a working integration quickly, but the output still required compiler checks, browser tests, and visual review. Review identified insufficient ground visibility and a demo overlay obscuring plants; both were adjusted. An initial capture contained long idle intervals, so the recording had to be corrected rather than submitted as-is.

The specification proposed Stable Diffusion/Midjourney asset references and Claude/ChatGPT shader assistance. The original mockup assets were not supplied. This implementation instead includes one newly generated OpenAI ground texture, with its exact prompt and provenance saved in public/textures/PROVENANCE.md. The background and plant geometry are procedural. No claim is made that the proposed image tools were used for this build. Runtime LLM naming was optional, is deferred, and is not required for the core scene.

AI provided useful starting code and an asset, while testing exposed practical integration and presentation issues. No manual-versus-AI time baseline was tracked, so time saved and debugging overhead cannot be quantified honestly. The recorded frame timings are measured evidence; statements about AI productivity are qualitative observations.

Roadmap to final

1. Profile the normal viewport on more desktop GPUs; investigate long frames, shader warm-up, point-light cost, and bloom resolution before making a stronger performance claim.
2. Add wind sway while keeping stems and attached leaves coherent. Introduce a movable light and controlled bending only after the generator stays stable.
3. Add day/night lighting, basic shadows, and the proposed Crystal Bloom, Spiral Vine, and Glowing Tree families as separate constrained rule sets.
4. Refine branch/leaf intersections, color ranges, texture repetition, and camera framing through visual review. Weather and falling petals remain stretch goals.
5. Publish the tested build, verify the public repository, upload the screen recording, and replace the pending links below before final submission.

Deliverables and links

Playable locally: npm ci, npm run dev. Production: npm run build, then npm run preview. Node.js 22.12+ and desktop WebGL2 are required. README.md contains the full instructions; no API key is needed.

Source remote: https://github.com/KartikP12s/Lumina-Garden

Live playable URL: PENDING deployment and verification. The included GitHub Pages workflow targets https://kartikp12s.github.io/Lumina-Garden/ after the local changes are pushed and Pages is configured. This target is not a verified live URL. GitHub's API confirms that the source repository is public and Pages is not yet enabled.

Video URL: PENDING YouTube/Vimeo upload. Local real-time recording: artifacts/LuminaGarden-Assignment3-Demo.webm. It shows a ground click, staged growth, text explanation overlays, bloom/light comparisons, and the 1/3/5-plant stress test. Videos are ignored by Git. The report PDF and benchmark evidence are stored under docs/.